// THE IDEA, IN FULL

Do the work.
Get paid on the spot.

Bounties for physical work, verified by photograph against a written acceptance test. A poster locks the payment and publishes the standard before anyone spends a minute. A worker photographs the place before and after. Several independent validators grade those two images against that same text, and the verdict and the payment are one transaction.

THE ONE SENTENCE VERSION

A poster writes an acceptance test, a worker submits a before and after photograph with a challenge code in frame, and the contract grades both images and pays.

Built on	GenLayer - Intelligent Contracts, Studio network
Status	Deployed, seeded and verified end to end
Contract	`0x7132E013d9b3e118319E92B8DAfFFB8De41c35bB`
Stack	Next.js 14 · genlayer-js · Python GenVM contract
Vision	Proven on chain - see page 5

Every number in this document is measured. Where something did not work, it says so and says what was done instead.

// CHAPTER 01

The problem

Anyone who coordinates physical work at scale pays twice: once for the work, and once for checking it. Field audits, installs, cleanups and shelf checks all end in a folder of photographs that nobody reads.

Crypto made paying a stranger easy and left verification exactly where it was. That is why physical bounty networks stay small. The worker feels it most: payment depends on a reviewer who may never look, and the person deciding is the person holding the money.

The cost is not just friction. Verification overhead is often larger than the task itself, which puts a floor under the size of job that can be crowdsourced at all. Below that floor, the work simply does not happen.

Who pays for this today

WHO	WHAT THEY NEED
DePIN networks	Proof that hardware was really installed where it was claimed
Brands and retail	Shelf presence and display checks, today done by expensive audit firms
Local groups	Cleanups, repairs and civic tasks funded by a pool, with proof attached
Field ops teams	A verification API they can point their own workforce at

Today versus here

STEP	HOW IT WORKS TODAY	HOW IT WORKS HERE
Grade a photo	A reviewer in an office days later, or a model owned by the payer	Independent validators grade the same two images against the same written test
Pay	Invoice, batch, thirty days	Payment lands in the same transaction as the verdict
Dispute	The poster decides and the worker has no recourse	The standard was public before the work began, and rejections say what to change
Detect reuse	Nobody checks whether the photo is from last month	A recycled photograph carries the wrong challenge code and fails

THE MOMENT THAT SELLS IT

A worker finishes a cleanup, photographs it with the code on a scrap of paper, and is paid before they walk to the next street. The receipt is public.

// CHAPTER 02

Why this needs GenLayer

This does not use a model as a backend. It uses one where a judgement has to be settled between parties who do not trust each other - which is the only thing a consensus layer is actually for.

A poster and a worker disagree about whether a job was done. Doing that with one model owned by the payer is exactly the arrangement workers already distrust. Here the standard is written down and made public before anyone spends time, several validators grade the same pair of photographs against that same text independently, and they must agree before a single coin moves.

The boundary, decided before any code was written

WHO OWNS IT	WHAT
Frontend	The camera, the checklist, uploads, the map, non-authoritative previews
The contract	The acceptance test, the challenge code, the grading, the reuse checks, the payout
Storage	The photographs - content addressed, so every validator provably grades identical bytes

What the platform is actually used for

PRIMITIVE	ITS JOB HERE
`exec_prompt(images=[a, b])`	Vision grading of the before and after pair
`web.request(url)`	Fetches content addressed images so every node grades identical bytes
`run_nondet_unsafe(l, v)`	Leader proposes, validators independently reach their own verdict, decisions are compared
`eq_principle` / LLM gate	Refuses an acceptance test too vague to grade, at posting time
Pillow, in the consensus block	Pre-flight checks on the pixels before paying for a grader
`@gl.public.write.payable`	Task funding - the reward is locked when the task is posted
`gl.Event`	TaskPosted / Claimed / Graded / Refused, for the indexer

THE LINE THAT MATTERS MOST

The images must be content addressed. If each node fetched a mutable url, the leader and the validators could be looking at different photographs, and the whole verification would be theatre.

// CHAPTER 03

Who owns the before frame

The poster, not the worker. This is the one design decision that changes what the product is.

If the worker supplies both frames they control the comparison entirely. They can photograph a mess, clear it, and be paid for work nobody needed; every check downstream - the acceptance test, the same-place judgement, the pre-flight - is then measured against a starting state the person being paid chose. That is the difference between grading work and grading a story about work.

The cost is real and is published on the site’s limits page rather than buried. The challenge code is issued at claim time, so it does not exist yet when the poster shoots. The code can only be checked in the worker’s frame. That is the weaker of the two properties, and staging is the more expensive fraud to be wrong about.

Two consequences follow in the contract. The before photograph is fetched and pre-flighted while the task is being posted, so nobody walks to a job that could never be graded. And the before frame’s content id is written into the reuse set at posting time, so handing the poster’s own file back as an after frame is caught by the same check as any other recycled photograph.

// CHAPTER 04

How it works, end to end

1 - Post, photograph, and fund

The poster writes an acceptance test that names observable things: *the bin area is empty, no bags remain against the wall, the ground is clear of loose litter, and both bins are upright with their lids closed.* They photograph how the place looks now, and send the reward plus the fee in the same transaction. The money is locked before any worker sees the task - and so is the starting state.

2 - The gate nobody expects

Before the task is created at all, the contract reads the acceptance test and decides whether it can be judged from two photographs. In the same round trip it opens the poster's photograph, so a task can never be funded with a before frame nobody could grade. *"Make sure the area is nice and clean and looks good when you finish"* is refused. A vague test poisons every submission made against it and the worker carries the cost, so this is the cheapest possible place to catch one.

3 - Claim

A worker takes the task. The contract issues a six character code derived deterministically from the task, the worker and the moment - so anyone auditing the record later can recompute it. The claim lasts as long as the poster chose when they funded the task, between ten minutes and a week, and ninety minutes when they did not choose.

4 - Photograph

They write the code on paper and photograph the finished work with it in frame. One frame, not two: the before frame is already on the task. The app re-encodes to a standard baseline JPEG, strips EXIF including any GPS they did not mean to publish, and puts it in content addressed storage.

5 - Pre-flight

The contract opens the worker's image before paying for a grader, the same way it opened the poster's at step 2. Unopenable, under 480px on the long edge, or shot into the sun - refused for the price of a decode, with a specific instruction rather than a verdict.

6 - Judge

Validators fetch the same bytes and grade them against the same text. They must agree on three things: the code is legible in the worker's frame, both frames show the same place, and the acceptance test passed. Reasons are never compared - two graders describe one photograph differently, and demanding identical prose would fail consensus on agreeing verdicts.

7 - Settle

Verdict and payment are one transaction. A public receipt is left behind: both photographs, the text they were graded against, the three judgements, and the reason given to the worker.

WHAT A REJECTION COSTS

Not the task. Most failures are lighting and framing, not fraud, so the claim stays with the worker and they can retake inside the window. And if they walk away, the task returns to the pool - a missed claim is not fraud either.

// CHAPTER 05

What was measured

Four things in this product were decided by measurement rather than argument. Two of them killed a feature that had already been built.

Vision works - and three traps that all report the same error

A probe contract on Studio, shown a photograph of a golden retriever holding a flower, answered **“golden retriever with a flower”**. Getting there meant clearing three separate causes that every one of them surfaces as an unexplained `INVALID_IMAGE`:

CAUSE	WHAT ACTUALLY HAPPENED
The fetch was not an image	Wikimedia answers 403 to a client with no User-Agent. A 126 byte text error page was handed to the model as a photograph. The contract now checks the status before trusting the body.
A JPEG with no JFIF header	Magic bytes `ffd8ffdb` is rejected outright, while the same subject as baseline JFIF works at 43KB *and* at 520KB. It is the container, not the size. Uploads are re-encoded.
A model that cannot see	Studio's router can hand the call to a text-only model that answers confidently anyway - the same photo returned “A white toilet bowl” and “No image provided” on different runs. The prompt now demands a `saw_images` flag and the contract refuses to act without it.

Perceptual matching was built, measured, and removed

Detecting a cropped or re-saved photograph looks like the obvious anti-fraud feature, so it was built - dHash plus LSH banding. Then it was measured, at 64 bits:

COMPARISON	DISTANCE
The same photograph, re-encoded at q55	8
The same place, photographed on another day	**2**
A different scene entirely	16 - 20

The populations overlap, and they overlap the wrong way round: honest repeat work at the same corner scores *closer* than actual reuse. Going to 256 bits made it worse. This is not a tuning problem - it is the domain. Every task in this product is a photograph of the same place, so “looks almost identical” is the normal case rather than the suspicious one.

And the failure is not symmetric: a false positive tells a worker who did the job that they are a fraud. So it does not vote. **The challenge code was always the real defence** - a recycled photograph carries the wrong one, and the vision call already checks for it.

Two bugs a fresh read found

BUG	CONSEQUENCE
A rejected task never returned to the pool	A worker rejected once and then gone left the task frozen for ever - unclaimable by anyone, reward locked until the poster noticed.
Overpaying a task burned the excess	Anything sent above the reward and fee was stranded: no refund path and no withdrawal path reached it.

Neither appeared in linting, type checking or any test - both lived in paths nothing exercised. Both are fixed and both now have regression tests.

// CHAPTER 06

What this cannot do

The product ships a page saying this, at `/limits`. If any other page seems to promise more, that page is the one that is true.

It cannot prove where a photograph was taken

No system can. A phone's reported coordinates can be changed. Nothing is ever marked location verified. What it does instead: the before frame comes from the poster rather than the worker, a challenge code issued at claim time that must be legible in the worker's frame, and a same-place check between the two. All three are in the contract, and stacking them still does not add up to a location proof.

It does not match photographs by how they look

Exact reuse is caught - the content hash and the content id of every accepted photograph are stored. A cropped or re-saved one is not, for the measured reason on the previous page.

The model can be wrong, and there is no appeal

Several validators grading the same evidence and agreeing is not the same as being right. Nothing in the contract can overturn a verdict once it is final, no account has the power to, and there is no human review step. What a rejected worker has instead is the rest of their window to retake, and a receipt carrying both photographs, the exact text and the three judgements, so anyone can check the verdict against the evidence. That is weaker than review by a person and it is the one that exists.

It does not know whether a task is safe to do

The gate at posting time asks one question only: could this acceptance test be graded from a photograph. Nothing reads it for private property, confrontation or hazardous material, so a dangerous but precisely written task passes. A posted task is not a vetted task.

Small tasks do not pay for themselves

A vision call with two images runs once per validator, which is the most expensive thing the contract does. Below roughly ten GEN a task costs more to settle than it is worth. There is no batching, so that floor is real rather than something a route builder works around later.

On Studio, the money does not actually move

Measured directly against the deployed contract: funding a task moved 19.08 GEN in correctly, and the refund took exactly 19.08 GEN out of the contract while the payee's balance did not change by a single wei. The contract is correct - Studio's ledger does not apply an emitted transfer to an ordinary account. The site says so wherever it claims payment. On a live network the same transaction pays.

// CHAPTER 07

Economics, and where it stands

How it makes money

LINE	WHAT IT IS
Take rate	Six percent of each paid bounty, charged to the poster. Set on chain.
Campaign plans	A monthly fee for bulk posting, coverage reporting and exports
Verification API	Other networks send an image pair and a test, and pay per verification
Assurance tier	A sample of tasks checked twice, sold as a guarantee

What it costs to run

One vision call with two images per submission, repeated per validator - the most expensive operation in the design, which is exactly what the pre-flight checks exist to avoid wasting. Rewards need to sit above roughly ten units.

The risk register

Every answer below is code that runs today. Where a risk is only partly answered it says so, because a register that lists intentions beside shipped mechanisms is not a register.

RISK	WHAT ANSWERS IT
Photo fraud	The poster owns the before frame, a claim-time challenge code, exact reuse caught by content hash and content id
Location claims	Never presented as proven, on any screen. Nothing more - there is no repeat verification pass
Worker safety	Not answered. The posting gate tests gradeability only, and does not read a task for danger
Vague tests	Refused on chain by a model before the task can be funded
Model bias	Not answered. There is no appeal and no audit pass. The whole judgement is published so a wrong one is checkable
A grader that cannot see	The saw_images flag - a blind model produces a retry, not a verdict

Where it stands today

Contract	33 methods - lint, validate and type check all clean
Deployed	`0x7132E013d9b3e118319E92B8DAfFFB8De41c35bB` on Studio
Seeded	Real tasks on chain, posted through the acceptance-test gate

Proven on chain	Posting, the vague-test refusal, underfunding, claiming, double-claim
Proven separately	Vision grading, via a dedicated probe contract
Not yet proven	A full passing submission - it needs two real photographs
Site	Seven routes, light and dark, both measured for contrast

Start narrow. One city, one task type, twenty workers - because a physical network that is thin everywhere is worth less than one that is dense in a single neighbourhood.

fieldwork.app · physical work, verified by photo